MASTER OF COMPUTER APPLICATIONS

PRACTICAL RECORD WORK
ON
20MCA132 – OBJECT ORIENTED PROGRAMMING LAB

Submitted by

NAME : DIYAKRISHNA T K

Reg. No. : VDA24MCA-2017

DEPARTMENT OF COMPUTER APPLICATIONS

COLLEGE OF ENGINEERING VATAKARA

(CAPE – GOVT. OF KERALA)

JUNE 2025

DEPARTMENT OF COMPUTER APPLICATIONS

COLLEGE OF ENGINEERING VATAKARA

CAPE(– GOVT. OF KERALA)

CERTIFICATE

Certified that this is the bonafide record work on the practical course **20MCA132 OBJECT ORIENTED PROGRAMMING LAB** done and prepared by **Ms. DIYAKRISHNA T K (Reg No.VDA24MCA-2017)**, 2nd Semester **MCA (2024-26 Batch)** student of Department of Computer Applications at College of Engineering Vadakara, in the partial fulfilment of the award of MCA Degree of APJ Abdul Kalam Technological University (KTU).

Date :

Faculty-in-Charge **Head of the Department**

(Office Seal)

Internal Examiners: **External Examiners:**

INDEX

SL.NO	EXPERIMENT	DATE	PAGE NO	REMARKS
1	Define a class ‘product’with data members pcode, pname and price. Create 3 objects of 1 the class and find the product having the lowest price.	06-02-2025	1	
2	Read 2 matrices from the console and perform 3 matrix addition.	07-02-2025	3	
3	Add complex numbers.	07-02-2025	6	
4	Read a matrix from the console and check 7 12-02-24 whether it is symmetric or not.	13-02-2025	8	
5	Create CPU with attribute price. Create inner class Processor (no. of cores, manufacturer) and static nested class RAM (memory, manufacturer). Create an object of CPU and print information of Processor and RAM.	13-02-2025	10	
6	Program to Sort strings	19-02-2025	12	
7	Search an element in an array	19-02-2025	14	
8	Perform string manipulations.	25-02-2025	16	
9	Program to create class for Employee having attributes eNo, eName eSalary. Read n employ information and Search for an 16 employee given eNo, using the concept of Array of Objects.	25-02-2025	18	
10	Area of different shapes using overloaded functions.	04-03-2025	20	
11	Program to create a class Employee and a subclass Teacher with additional details. Use an array of objects to display details of N teachers.	04-03-2025	22	

12	Program to implement multilevel inheritance using classes Person, Employee, and Teacher with an array of objects to display details of N teachers.	05-03-2025	25	
13	Write a program has class Publisher, Book, Literature and Fiction. Read the information 27 and print the details of books from either the category, using inheritance.	15-03-2025	28	
14	Create classes Student and Sports. Create another class Result inherited from Student and Sports. Display the academic and sports score of a student.	20-03-2025	31	
15	Create an interface having prototypes of functions area() and perimeter(). Create two classes Circle and Rectangle which implements the above interface. Create a menu driven program to find area and perimeter of objects.	20-03-2025	34	
16	Prepare bill with the given format using calculate method from interface. Order No.	25-03-2025	37	
17	Create a Graphics package that has classes and interfaces for figures Rectangle, Triangle,Square and Circle. Test the package by finding the area of these figures.	12-04-2025	40	
18	Create an Arithmetic package that has classes and interfaces for the 4 basic arithmeticoperations. Test the package by implementing all operations on two given numbers	13-04-2025	42	

PROGRAM NO 1

AIM:

Define a class ‘product’with data members pcode, pname and price. Create 3 objects of the class and find the product having the lowest price.

ALGORITHM:

1. Start
2. Define a class `Product` with:
   -Data members: `pcode`, `pname`, `price`
   -Constructor to initialize the data members
3. In the `main` method:
   1. Create three `Product` objects:
      - `p1` with values: code = 1, name = "Laptop", price = 43000
      - `p2` with values: code = 2, name = "TV", price = 20000
      - `p3` with values: code = 3, name = "Mobile", price = 18000
4. Compare the prices of the three products:
   -If `p1.price` is less than both `p2.price` and `p3.price`, then
   -Print that `p1` has the lowest price
   -Else if `p2.price` is less than both `p1.price` and `p3.price`, then:
   -Print that `p2` has the lowest price
   -Else:
   -Print that `p3` has the lowest price
5. End

SOURCE CODE

```
class Product{
  int pcode;
   String pname;
   int price;

        Product(int pcode,String pname,int price){
                this.pcode=pcode;
                this.pname=pname;
               this.price=price;
                }
         public static void main(String[] args){
               Product p1= new Product(1,"Laptop",43000);
               Product p2= new Product(2,"TV",20000);
                Product p3= new Product(3,"mobile",18000);

                if(p1.price<p2.price && p1.price<p3.price){
                        System.out.println("lowest price is "+p1.pname+" with cost
               "+p1.price);
                        }else if(p2.price<p1.price && p2.price<p3.price){
```

```
            System.out.println("lowest price is "+p2.pname+" with cost "+p2.price);
            }else{
             System.out.println("lowest price is "+p3.pname+" with cost "+p3.price);
            }
    }
}
```

Output:

```
PS D:\MCA\sem2\Java> javac Product.java
PS D:\MCA\sem2\Java> java Product
code:1  name:Laptop     price:43000
code:2  name:TV price:20000
code:3  name:mobile     price:18000
lowest price is mobile with cost 18000
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 2

AIM:

Read 2 matrices from the console and perform matrix addition.

ALGORITHM:

1.Start
2.Define a class AddMatrix with:
-Integer variables r, c for number of rows and columns.
-A 2D integer array arr[10][10] to store matrix elements.
-A method get() to input matrix dimensions and elements.
-A static method sum(AddMatrix c1, AddMatrix c2) to add two matrices.
-A method display() to print the matrix.
3.Define method get():
-Prompt the user to enter number of rows r and columns c.
-Read values into the arr matrix using nested loops.
4.Define static method sum(AddMatrix c1, AddMatrix c2):
-Create a new AddMatrix object temp.
-Check if c1 and c2 have the same dimensions.
-If they do:
-Set temp.r = c1.r and temp.c = c1.c.
-Use nested loops to add corresponding elements:
-temp.arr[i][j] = c1.arr[i][j] + c2.arr[i][j]
-If not, print "Addition not possible".
-Return temp.
5.Define method display():
-Use nested loops to print each element of the matrix.
6.In the main() method:
-Create objects m1, m2, and result of class AddMatrix.
-Call m1.get() and m2.get() to read matrices.
-Call sum(m1, m2) and store the result in result.
-Print the result using result.display().
7. End

SOURCE CODE

```
import java.util.Scanner;
class AddMatrix{
  int r,c;
int[][] arr=new int[10][10];
public void get(){
Scanner sc= new Scanner(System.in);
 System.out.println("enter the number of rows");
  this.r = sc.nextInt();
 System.out.println("enter the number of columns");
```

```
        this.c = sc.nextInt();
        System.out.println("enter the matrix");
         for(int i=0;i<this.r;++i){
          for(int j=0;j<this.c;++j){
         this.arr[i][j]=sc.nextInt();
     }
    }
  }
  public static AddMatrix sum(AddMatrix c1, AddMatrix c2) {
     AddMatrix temp = new AddMatrix();
     if (c1.r == c2.r && c1.c == c2.c) {
         temp.r =c1.r;
         temp.c= c1.c;
         for (int i = 0; i < c1.r; i++) {
             for (int j = 0; j < c1.c; j++) {
                 temp.arr[i][j] = c1.arr[i][j]+c2.arr[i][j];
             }
         }
     }
     else {
         System.out.println("Addition not possible ");
     }
     return temp;
  }
  public void display(){
    for(int i=0;i< this.r;++i){
      for(int j=0;j<this.c;++j){
        System.out.print(this.arr[i][j]+" ");
      }
      System.out.println();
    }
  }
  public static void main(String[] args){
    AddMatrix m1=new AddMatrix();
    AddMatrix m2 = new AddMatrix();
    AddMatrix result = sum(m1,m2);
    m1.get();
    m2.get();
    result=sum(m1,m2);
    System.out.println("addition of matrix");
    result.display();

  }
}
```

Output:

```
PS D:\MCA\sem2\Java> javac AddMatrix.java
PS D:\MCA\sem2\Java> java AddMatrix
enter the number of rows
2
enter the number of columns
2
enter the matrix
1
2
3
4
enter the number of rows
2
enter the number of columns
2
enter the matrix
4
5
6
7
addition of matrix
5 7
9 11
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 3

AIM:

Add 2 complex numbers

ALGORITHM:

1. Start
2. Define a class Complex with:
   1. Data members:
      real → to store real part of complex number
      imaginary → to store imaginary part of complex number
   2. A constructor to initialize the data members with user-defined values.
3. Define a static method `sum(Complex c1, Complex c2):
   1. Create a new Complex object temp initialized with (0, 0).
   2. Set temp.real = c1.real + c2.real.
   3. Set temp.imaginary = c1.imaginary + c2.imaginary.
   4. Return temp.
4. In the `main()` method:
   1. Create object first with values (1, 2).
   2. Create object second with values (32, 22).
   3. Call the sum(first, second)` method and store result in object temp.
   4. Print the result in the format: real + imaginary i.
5. End

SOURCE CODE

```
public class Complex {
 int real;
 int imaginary;

 public Complex(int real,int imag){
     this.real = real;
     this.imaginary = imag;
 }
 public static Complex sum(complex c1,complex c2){
     complex temp = new Complex(0,0);
     temp.real = c1.real + c2.real;
     temp.imaginary = c1.imaginary + c2.imaginary;
     return temp;
 }
 public static void main(String []args){
     Complex first = new Complex(1,2);
     Complex second = new Complex(32,22);
     Complex temp = sum(first,second);
     System.out.println("Sum is : " + temp.real + "+" +temp.imaginary + "i");
 }
}
```

Output:

```
PS D:\MCA\sem2\Java> javac Complex.java
PS D:\MCA\sem2\Java> java Complex
Sum is : 33+24i
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 4

AIM:

Read a matrix from the console and check whether it is symmetric or not.

ALGORITHM:

1. Start
2. Define a class Symmetric with:
    1. Integer variables r and c for number of rows and columns.
    2. A 2D integer array arr to store matrix elements.
    3. A method get() to input matrix elements.
    4. A method isSym() to check for symmetry.
3. Define method get():
    1. Prompt the user to input the number of rows r and columns c.
    2. Create a matrix of size r x c.
    3. Use nested loops to input the elements of the matrix from the user.
4. Define method isSym():
    1. Initialize a flag variable to 0.
    2. Use nested loops to compare element arr[i][j] with arr[j][i] for all i and j.
    3. If any mismatch is found:
        Set flag = 1
        Break out of the loop.
    4. After checking:
        If flag == 0, print "this is symmetric"
        Else, print "this is not symmetric"
5. In the main() method:
    1. Create an object matrix of class Symmetric.
    2. Call matrix.get() to input matrix.
    3. Call matrix.isSym() to check symmetry and display result.
6. End

SOURCE CODE

```
import java.util.Scanner;
public class Symmetric{
  int r,c;
  int[][] arr;
  public void get(){
    Scanner sc = new Scanner(System.in);
    System.out.println("enter the number of row");
    this.r= sc.nextInt();
    System.out.println("enter the number of columns");
    this.c=sc.nextInt();
    arr=new int[r][c];
    System.out.println("enter the matrix");
    for(int i=0; i<this.r; ++i){
      for(int j=0; j<this.c; ++j){
```

```
this.arr[i][j]=sc.nextInt();
        }
      }
    }

    public void isSym(){
      int flag=0;
       for(int i=0;i<this.r;++i){
         for(int j=0;j<this.c;++j){
            if(this.arr[i][j] != this.arr[j][i]){
               flag=1;
               break;
            }
              }
            if(flag==1){
              break;
            }
       }
       if(flag==0){
          System.out.println("this is symmetric");
         }else{
          System.out.println("this is not symmetric");
         }
    }

    public static void main(String args[]){
      Symmetric matrix = new Symmetric();
      matrix.get();
      matrix.isSym();
    }
  }
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac Symmetric.java
PS D:\MCA\sem2\Java> java Symmetric
enter the number of row
2
enter the number of columns
2
enter the matrix
1
2
2
3
this is symmetric
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 5

AIM:
Create CPU with attribute price. Create inner class Processor (no. of cores, manufacturer) and static nested class RAM (memory, manufacturer). Create an object of CPU and print information of Processor and RAM.

ALGORITHM:

1. Start
2. Define a base class Cpu
   2.1. Declare data member price
   2.2. Define constructor to initialize price
   2.3. Define method display() to show CPU price
3. Define a derived class Processor extending Cpu
   3.1. Declare data members cores and manufacturer
   3.2. Define constructor with parameters for price, cores, and manufacturer
   3.3. Use super(price) to call the base class constructor
   3.4. Initialize cores and manufacturer
   3.5. Define method displayProcessor() to:
      3.5.1. Call display() to print CPU price
      3.5.2. Print number of cores
      3.5.3. Print processor manufacturer
4. Define another derived class Ram extending Cpu
   4.1. Declare data members memory and manufacturer
   4.2. Define constructor with parameters for price, memory, and manufacturer
   4.3. Use super(price) to call base class constructor
   4.4. Initialize memory and manufacturer
   4.5. Define method displayRam() to:
      4.5.1. Print memory size
      4.5.2. Print RAM manufacturer
5. In the main() method:
   5.1. Create an object processor of class Processor with values (23000, 8, "Intel")
   5.2. Create an object ram of class Ram with values (23000, 16, "Corsair")
   5.3. Call processor.displayProcessor()
   5.4. Call ram.displayRam()
6. End

SOURCE CODE

```
class Cpu {
    int price;
    public Cpu(int p) {
        this.price = p;
    }
    void display() {
```

```
        System.out.println("Price of CPU : " + this.price);
}
  class Processor{
    int cores;
    String manufacture;

    Processor(int core, String manf) {
      this.cores = core;
      this.manufacture = manf;
    }
    void display() {
      System.out.println("No of Cores : " + this.cores);
      System.out.println("Processor manufactures : " + this.manufacture);
    }
  }
  static class Ram {
    int memory;
    String manufacture;

    Ram(int n, String m) {
      this.memory = n;
      this.manufacture = m;
    }
    void display() {
      System.out.println("Memory Size : " + this.memory);
      System.out.println("Memory manufactures : " + this.manufacture);
    }
  }
  public static void main(String[] args) {
    Cpu intel = new Cpu(23000);
    Cpu.Processor p=  intel.new Processor(8,"Intel");
    Cpu.Ram ram = new Ram(64, "Asus");
    intel.display();
    p.display();
    ram.display();
  }}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac Cpu.java
PS D:\MCA\sem2\Java> java Cpu
Price of CPU : 23000
No of Cores : 8
Processor manufactures : Intel
Memory Size : 64
Memory manufactures : Asus
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 6

AIM:

Program to Sort strings

ALGORITHM:

1. Start
2. Create a Scanner object to read input from the user.
3. Prompt the user to enter the number of strings (n).
4. Read the integer n from the user.
5. Create a string array of size n to store the input strings.
6. Prompt the user to enter n strings.
7. Use a loop to read n strings and store them in the array:
   -For i from 0 to n-1:
   Read string and store in strings[i].
8. Sort the string array using `Arrays.sort(), which arranges strings in lexicographical order (dictionary order).
9. Display the sorted list of strings** using a loop:
   For each string in the sorted array, print the string.
10. Close the Scanner object.
11. End

SOURCE CODE

```
import java.util.Arrays;
import java.util.Scanner;
public class StringSort
{
public static void main(String[] args){
Scanner scanner = new Scanner(System.in);
System.out.print("Enter the number of strings:\n");
int n = scanner.nextInt();
String[] strings = new String[n];
System.out.println("Enter the strings:");
for (int i = 0; i < n; i++)
{
strings[i] = scanner.next();
}
Arrays.sort(strings);
System.out.println("\nSorted strings:");
for (String str : strings)
{
System.out.println(str);
}
scanner.close();
}
}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac Sort.java
PS D:\MCA\sem2\Java> java Sort
Enter the number of strings:
3
Enter the strings:
Rahul
Anil
Devu

Sorted strings:
Anil
Devu
Rahul
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 7

AIM:

Search an element in an array.

ALGORITHM:

1. Start
2. Declare an array arr[10] and a variable count to store the number of elements
3. Define method get() to input array elements:
   3.1. Create a Scanner object.
   3.2. Prompt the user to enter the number of elements (count).
   3.3. Read the count.
   3.4. Prompt the user to enter array elements.
   3.5. Use a loop to read each element and store in arr[i] for i = 0 to count - 1.
4. Define method itemSearch() to search for an element:
   4.1. Prompt the user to enter the search element n.
   4.2. Initialize flag = 0.
   4.3. Use a loop from i = 0 to count - 1:
        - If arr[i] == n:
          - Set flag = 1
          - Break the loop
   4.4. If flag == 1:
        - Print that the element is found at position i + 1
   4.5. Else:
        - Print "element not found"
5. In the main() method:
   5.1. Create an object s1 of class search.
   5.2. Call s1.get() to input array elements.
   5.3. Call s1.itemSearch() to perform the search.
6. End

SOURCE CODE

```
import java.util.*;
public class search{
  int[] arr = new int[10];
  int count;

  public void get(){
    Scanner sc = new Scanner(System.in);
    System.out.println("enter the count of array: ");
    this.count = sc.nextInt();
    System.out.println("enter the array elements");
    for(int i=0;i<this.count;++i){
      this.arr[i]=sc.nextInt();
     }
   }
   public void itemSearch(){
```

```
    int n,flag=0,i;
     Scanner sc = new Scanner(System.in);
     System.out.println("enter the element you want to search");
     n=sc.nextInt();
     for(i=0;i<this.count;++i){
        if(this.arr[i]==n){
            flag=1;
            break;
          }
      }
      if(flag==1){
       System.out.println("element is found at position "+(i+1));
      }else{
       System.out.println("element is not found");
      }
    }

    public static void main(String[] args){
       search s1 = new search();
       s1.get();
       s1.itemSearch();
    }
}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac search.java
PS D:\MCA\sem2\Java> java search
enter the count of array:
5
enter the array elements
3
5
7
2
1
enter the element you want to search
7
element is found at position 3
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 8

AIM:

Perform string manipulations.

ALGORITHM:

1. Start
2. Initialize a string variable str with the value "Hello, World!"
3. Find the length of the string using length() method
   - Store the result in length
   - Print the length
4. Convert the string to uppercase using toUpperCase()
   - Store in uppercase
   - Print the result
5. Convert the string to lowercase using toLowerCase()
   - Store in lowercase
   - Print the result
6. Replace all occurrences of the character 'o' with 'x' using replace()

   - Store in replaced
   - Print the result
7. Extract a substring from index 7 to 11 (end index is exclusive) using substring(7, 12)
   - Store in substring
   - Print the result
8. Check if the string starts with "Hello" using startsWith()
   - Store result in startsWith
   - Print the result
9. Check if the string ends with "World!" using endsWith()
   - Store result in endsWith
   - Print the result
10. Trim leading and trailing spaces from the string " Hello, World! " using trim()
    - Store in trimmed
    - Print the result
11. End

SOURCE CODE

```
public class StringManipulation
{
public static void main(String[] args)
{
String str = "Hello, World!";
int length = str.length();
System.out.println("Length: " + length);
// Convert to uppercase
String uppercase = str.toUpperCase();
System.out.println("Uppercase: " + uppercase);
```

```
// Convert to lowercase
String lowercase = str.toLowerCase();
System.out.println("Lowercase: " + lowercase);
// Replace characters
String replaced = str.replace('o', 'x');
System.out.println("Replaced: " + replaced);
// Extract substring
String substring = str.substring(7, 12);
System.out.println("Substring: " + substring);
// Check if a string starts with a specific prefix
boolean startsWith = str.startsWith("Hello");
System.out.println("Starts with 'Hello': " + startsWith);
// Check if a string ends with a specific suffix
boolean endsWith = str.endsWith("World!");
System.out.println("Ends with 'World!': " + endsWith);
// Trim leading and trailing whitespace
String trimmed = " Hello, World! ".trim();
System.out.println("Trimmed: " + trimmed);
}
}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac StringManipulation.java
PS D:\MCA\sem2\Java> java StringManipulation
Length: 13
Uppercase: HELLO, WORLD!
Lowercase: hello, world!
Replaced: Hellx, Wxrld!
Substring: World
Starts with 'Hello': true
Ends with 'World!': true
Trimmed: Hello, World!
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 9

AIM:

Program to create a class for Employee having attributes eNo, eName eSalary. Read n employ information and Search for an employee given eNo, using the concept of Array of Objects.

ALGORITHM:

1. Start
2. Define a class Employee with data members:
   - eno (Employee ID)
   - ename (Employee Name)
   - salary (Employee Salary)
3. Define the read() method to:
   - Prompt user to input eno, ename, and salary
   - Read and store these values
4. Define the display() method to:
   - Print eno, ename, and salary
5. In the main() method:
   - Set n = 3 (number of employees)
   - Create an array emp[] of type Employee with n elements
6. For i = 0 to n-1:
   - Initialize emp[i] as a new Employee object
   - Call emp[i].read() to read details
7. Print "Search"
8. Repeat infinitely (or until externally stopped):
   - Prompt user to enter an Employee ID
   - Read the input as no
   - For i = 0 to n-1:
     - If emp[i].eno equals no:
       - Call emp[i].display() to show details
       - Break the loop
9. End

SOURCE CODE

```
import java.util.*;
class employee{
   int eno;
   String ename;
   int salary;
   public void read(){
    Scanner sc = new Scanner(System.in);
    System.out.println("enter the id:");
    this.eno = sc.nextInt();
    sc.nextLine();
    System.out.println("enter the employee name:");
    this.ename = sc.nextLine();
System.out.println("enter the salary:");
    this.salary = sc.nextInt();
```

```
}
  public void display(){
    System.out.println("employee id: "+this.eno);
    System.out.println("employee name: "+ this.ename);
    System.out.println("employee salary: "+this.salary);
   }
   public static void main(String[] args){
     Scanner sc =new Scanner(System.in);
     int i,n;
     System.out.println("enter no of employees");
     n = sc.nextInt();
     employee emp[] = new employee[n];
     for(i=0;i<n;i++){
        emp[i] = new employee();
        emp[i].read();
     }
     System.out.println("Search");
     while(true){
        System.out.print("Enter ID to search (or -1 to exit): ");
        int no = sc.nextInt();
        if(no == -1) break;
           boolean found = false;
        for(i=0;i<n;i++){
           if(emp[i].eno == no){
              emp[i].display();
              found = true;
              break;
           } }
        if(!found){
           System.out.println("Employee not found.");
           break;
        }} }}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac employee.java
PS D:\MCA\sem2\Java> java employee
enter no of employees
2
enter the id:
101
enter the employee name:
rahbar
enter the salary:
50000
enter the id:
104
enter the employee name:
sam
enter the salary:
45000
Search
Enter ID to search (or -1 to exit): 104
employee id: 104
employee name: sam
employee salary: 45000
Enter ID to search (or -1 to exit): -1
PS D:\MCA\sem2\Java>
```

Result: Successfully executed

PROGRAM NO 10

AIM:

Area of different shapes using overloaded functions

ALGORITHM:

1. Start
2. Define a class Area
3. Inside the class, define overloaded methods named area:
   3.1 For Circle:
   - Input: r (radius, double)
   - Compute area as π * r * r
   - Return result

   3.2 For Rectangle:
   - Input: l (length, double), w (width, double)
   - Compute area as l * w
   - Return result

   3.3 For Triangle:
   - Input: b (base, int), h (height, int)
   - Compute area as -0.5 * b * h
   - Return result

   For Square:
   - Input: l (side, int)
   - Compute area as l * l
   - Return result
4. In the main() method:
   - Create an object obj of class Area
   - Call obj.area(5.0) → prints area of circle
   - Call obj.area(3.0, 4.0) → prints area of rectangle
   - Call obj.area(4, 3) → prints area of triangle
   - Call obj.area(5) → prints area of square
5. End

SOURCE CODE

```
public class Area{
  //circle
  public double area(double r){
     return Math.PI*r*r;
    }
   // rectangle
  public double area(double l,double w){
      return l*w;
     }
   // triangle
  public double area(int b,int h){
     return 0.5*b*h;
```

```
}
 // square
 public int area(int l){
    return l*l;
   }
 public static void main(String[] args){
    Area obj = new Area();
    System.out.println("area of circle r=5: "+obj.area(5.0));
    System.out.println("area of rectangle(3,4): "+obj.area(3.0,4.0));
    System.out.println("area of triangle(4,3): "+obj.area(4,3));
    System.out.println("area of square(8): "+obj.area(5));
  }
}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac Area.java
PS D:\MCA\sem2\Java> java Area
area of circle r=5: 78.53981633974483
area of rectangle(3,4): 12.0
area of triangle(4,3): 6.0
area of square(8): 25
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 11

AIM:

Create a class 'Employee' with data members Empid, Name, Salary, Address and constructors to initialize the data members. Create another class 'Teacher' that inherit the properties of class employee and contain its own data members department, Subjects taught and constructors to initialize these data members and also include display function to display all the data members. Use array of objects to display details of N teachers.

ALGORITHM:

1. Start
2. Define the class Employee with:
   - Data members: id, name, sal, address
   - Constructor to initialize data members
   - Method display() to print employee details
3. Define the class teacher that extends Employee:
   - Additional data members: dept, sub
   - Constructor to initialize both base and derived class data
   - Override `display()` to include teacher-specific data
4. In the main() method (inside class co3_2):
   - Create a Scanner object to read input
   - Prompt the user to enter number n (number of teachers)
   - Create an array t of teacher objects of size n
5. Repeat for each teacher from i = 0 to n - 1:
   - Prompt and read values: id, name, salary, address, department, subject
   - Create a new teacher object using these values and store in array t[i]
6. Display the details of all teachers:
   - Loop through the array and call display() on each t[i]
7. End

SOURCE CODE

```
  import java.util.Scanner;
class Employee {
    int id;
    String name;
    int sal;
    String address;
 Employee(int id, String name, int sal, String address) {
        this.id = id;
        this.name = name;
        this.sal = sal;
        this.address = address;
    }
public void display() {
        System.out.println("Employee id: " + this.id);
```

```
        System.out.println("Employee name: " + this.name);
        System.out.println("Employee salary: " + this.sal);
        System.out.println("Employee address: " + this.address);
    }
}
class teacher extends Employee {
    String dept;
    String sub;

    teacher(int id, String name, int sal, String address, String dept, String sub) {
        super(id, name, sal, address);
        this.dept = dept;
        this.sub = sub;
    }
    public void display() {
        super.display();
        System.out.println("Department: " + this.dept);
        System.out.println("Subject: " + this.sub);
    }
}
public class co3_2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of teachers: ");
        int n = sc.nextInt();
        sc.nextLine();
        teacher[] t = new teacher[n];

        for (int i = 0; i < n; i++) {
            System.out.println("Enter details for teacher " + (i + 1) + ":");
            System.out.print("ID: ");
            int id = sc.nextInt();
            sc.nextLine();
            System.out.print("Name: ");
            String name = sc.nextLine();
            System.out.print("Salary: ");
            int sal = sc.nextInt();
            sc.nextLine();
            System.out.print("Address: ");
            String address = sc.nextLine();
            System.out.print("Department: ");
            String dept = sc.nextLine();
            System.out.print("Subject: ");
            String sub = sc.nextLine();
            t[i] = new teacher(id, name, sal, address, dept, sub);
        }
        System.out.println("\nTeacher Details:\n");
        for (int i = 0; i < n; i++) {
```

```
            t[i].display();
        }
    }
}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac co3_2.java
PS D:\MCA\sem2\Java> java co3_2
Enter number of teachers: 2
Enter details for teacher 1:
ID: 101
Name: arya
Salary: 35000
Address: calicut
Department: cse
Subject: java
Enter details for teacher 2:
ID: 110
Name: ram
Salary: 40000
Address: vadakara
Department: mca
Subject: dbms

Teacher Details:

Employee id: 101
Employee name: arya
Employee salary: 35000
Employee address: calicut
Department: cse
Subject: java
Employee id: 110
Employee name: ram
Employee salary: 40000
Employee address: vadakara
Department: mca
Subject: dbms
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 12

AIM:

Create a class 'Person' with data members Name, Gender, Address, Age and a constructor to initialize the data members and another class 'Employee' that inherits the properties of class Person and also contains its own data members like Empid, Company_name, Qualification, Salary and its own constructor. Create another class 'Teacher' that inherits the properties of class Employee and contains its own data members like Subject, Department, Teacherid and also contain constructors and methods to display the data members. Use array of objects to display details of N teachers.

ALGORITHM:

1. Start
2. Define the class Person:
   - Data members: name, gender, address, age
   - Constructor to initialize these data members
3. Define the class Employee that extends Person:
   - Additional data members: empId, companyName, qualification, salary
   - Constructor to initialize both Person and Employee data members using super() for inheritance
4. Define the class Teacher that extends Employee:
   - Additional data members: subject, department, teacherId
   - Constructor to initialize both Person, Employee, and Teacher data members using super()
   - Method display() to print details of the teacher, including all inherited attributes from Person and Employee
5. In the main() method (in class co3_3):
   - Create an array teachers of size 2 to store Teacher objects
   - Create two Teacher objects with appropriate data and assign to teachers[0] and teachers[1]
   - Call display() method on both teacher objects to print the details
6. End

SOURCE CODE

```
class Person {
    String name;
    String gender;
    String address;
    int age;
    Person(String name, String gender, String address, int age) {
        this.name = name;
        this.gender = gender;
        this.address = address;
        this.age = age;
    }
}
```

```
class Employee extends Person {
    int empId;
   String companyName;
   String qualification;
   double salary;

   Employee(String name, String gender, String address, int age, int empId,
           String companyName, String qualification, int salary) {
      super(name, gender, address, age);
      this.empId = empId;
      this.companyName = companyName;
      this.qualification = qualification;
      this.salary = salary;
   }
}
class Teacher extends Employee {
   String subject;
   String department;
   String teacherId;

   Teacher(String name, String gender, String address, int age, int empId,
      String companyName, String qualification,int salary,
      String subject, String department, String teacherId) {

      super(name, gender, address, age, empId, companyName, qualification, salary);
      this.subject = subject;
      this.department = department;
      this.teacherId = teacherId;
   }

   void display() {
      System.out.println("---------------");
      System.out.println("Name: " + name);
      System.out.println("Gender: " + gender);
      System.out.println("Address: " + address);
      System.out.println("Age: " + age);
      System.out.println("Emp ID: " + empId);
      System.out.println("Company: " + companyName);
      System.out.println("Qualification : " + qualification);
      System.out.println("Salary: " + salary);
      System.out.println("Subject: " + subject);
      System.out.println("Department: " + department);
      System.out.println("Teacher ID: " + teacherId);
   }
}
public class co3_3{
   public static void main(String[] args) {
      Teacher[] teachers = new Teacher[2];
```

```
 teachers[0]= new
Teacher("amal","male","vadakara",32,1001,"eduport","msc",45000,"maths","science","T101
");
        teachers[1]= new
Teacher("shahana","female","calicut",29,1002,"xylem","phd",50000,"physics","science","T1
03");
        teachers[0].display();
        teachers[1].display();
    }
}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac co3_3.java
PS D:\MCA\sem2\Java> java co3_3
---------------
Name: amal
Gender: male
Address: vadakara
Age: 32
Emp ID: 1001
Company: eduport
Qualification : msc
Salary: 45000.0
Subject: maths
Department: science
Teacher ID: T101
---------------
Name: shahana
Gender: female
Address: calicut
Age: 29
Emp ID: 1002
Company: xylem
Qualification : phd
Salary: 50000.0
Subject: physics
Department: science
Teacher ID: T103
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 13

AIM:

Write a program has class Publisher, Book, Literature and Fiction. Read the information and print the details of books from either the category, using inheritance.

ALGORITHM:

1. Start
2. Define the Publisher class:
   - Data member: publisherName
   - Constructor to initialize publisherName
3. Define the Book class that extends Publisher:
   - Additional data members: title, author
   - Constructor to initialize publisherName from Publisher class and title, author for Bookclass
4. Define the Literature class that extends Book:
   - Constructor to initialize publisherName, title, and author by calling the super() constructor
   - Method display() to print the details specific to the Literature category (Category, Title, Author, Publisher)
5. Define the Fiction class that extends Book:
   - Constructor to initialize publisherName, title, and author by calling the super() constructor
   - Method display() to print the details specific to the Fiction category (Category, Title, Author, Publisher)
6. In the main() method (in class BookStore):
   - Prompt user to enter the number of Literature books, read the number, and store them in literature Books array
   - For each Literature book, prompt user to enter the title, author, and publisher, then create a Literature object and store it in literatureBooks
   - Prompt user to enter the number of Fiction books, read the number, and store them in fiction Books array
   - For each Fiction book, prompt user to enter the title, author, and publisher, then create a Fiction object and store it in fictionBooks
7. Display details of all Literature books:
   - Loop through literatureBooks and call display() method for each book
8. Display details of all Fiction books:
   - Loop through fictionBooks and call display() method for each book
9. End

SOURCE CODE

```
import java.util.Scanner;
class Publisher {
   String publisherName;
   Publisher(String publisherName) {
      this.publisherName = publisherName;

}}
```

```
class Book extends Publisher {
    String title;
    String author;
    Book(String publisherName, String title, String author) {
        super(publisherName);
        this.title = title;
        this.author = author;
    }
}
class Literature extends Book {
    Literature(String publisherName, String title, String author) {
        super(publisherName, title, author);
    }
    void display() {
        System.out.println("Category: Literature");
        System.out.println("Title    : " + title);
        System.out.println("Author   : " + author);
        System.out.println("Publisher: " + publisherName);
        System.out.println("---------------------------");
    }
}
class Fiction extends Book {
    Fiction(String publisherName, String title, String author) {
        super(publisherName, title, author);
    }
    void display() {
        System.out.println("Category: Fiction");
        System.out.println("Title    : " + title);
        System.out.println("Author   : " + author);
        System.out.println("Publisher: " + publisherName);
        System.out.println("---------------------------");
    }
}
public class BookStore {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of Literature books: ");
        int n = sc.nextInt();
        sc.nextLine();
        Literature[] literatureBooks = new Literature[n];
        for (int i = 0; i < n; i++) {
            System.out.println("\nEnter details for Literature book " + (i + 1));
            System.out.print("Title: ");
            String title = sc.nextLine();
            System.out.print("Author: ");
            String author = sc.nextLine();
           System.out.print("Publisher: ");
            String publisher = sc.nextLine();

           literatureBooks[i] = new Literature(publisher, title, author);
```

```
        }
        System.out.print("\nEnter number of Fiction books: ");
        int n1 = sc.nextInt();
        sc.nextLine();
        Fiction[] fictionBooks = new Fiction[n1];
        for (int i = 0; i < n1; i++) {
            System.out.println("\nEnter details for Fiction book " + (i + 1));
            System.out.print("Title: ");
            String title = sc.nextLine();
            System.out.print("Author: ");
            String author = sc.nextLine();
            System.out.print("Publisher: ");
            String publisher = sc.nextLine();
            fictionBooks[i] = new Fiction(publisher, title, author);
        }
        System.out.println("\n------ Literature Books-----");
        for (Literature book : literatureBooks) {
            book.display();
        }
        System.out.println("\n----Fiction Books----");
        for (Fiction book : fictionBooks) {
            book.display();
        }
        sc.close();
    }
}
```

OUTPUT:

```
Enter number of Literature books: 2

Enter details for Literature book 1
Title: book1
Author: author1
Publisher: publisher1

Enter details for Literature book 2
Title: book2
Author: author2
Publisher: publisher2

Enter number of Fiction books: 1

Enter details for Fiction book 1
Title: book3
Author: author3
Publisher: publisher2

------ Literature Books-----
Category: Literature
Title     : book1
Author    : author1
Publisher: publisher1
---------------------------
Category: Literature
Title     : book2
Author    : author2
Publisher: publisher2
---------------------------

----Fiction Books----
Category: Fiction
Title     : book3
Author    : author3
Publisher: publisher2
---------------------------
```

Result:

Successfully executed

PROGRAM NO 14

AIM:

Create classes Student and Sports. Create another class Result inherited from Student and Sports. Display the academic and sports score of a student.

ALGORITHM:

1. Start
2. Define an Interface Sports
   Declare two abstract methods:
   - sports_getData()
   - sports_dispData()
3. Define the Student class
   - Data Members: name, roll_no, score
   - Constructor:
     - Accepts name, roll_no, and score (academic score)
     - Initializes the data members
   - Method display():
     - Displays the student's name, roll number, and academic score
4. Define the Result class that extends Student and implements Sports interface
   - Data Member: score (sports score)
   - Constructor:
     -Calls the superclass constructor to initialize student data
   -Method sports_getData():
     -Prompts the user to enter the sports score
     -Reads and stores it
   -Method sports_dispData():
     -Displays the sports score
5. In the main() method (class co3_5):
   - Create a Scanner object
   - Read student name, roll number, and academic score from the user
   - Create an object of Result class with the entered data
   - Call sports_getData() to input sports score
   - Call display() to show academic details
   - Call sports_dispData() to show sports score
6. End

SOURCE CODE:

```
import java.util.*;
import java.io.*;
interface Sports {
    public void sports_getData();
    public void sports_dispData();
}
class Student {
    String name;
```

```
int roll_no;
   int score;

   Student(String name,int roll, int m1) {
      this.name = name;
      this.roll_no = roll;
      this.score = m1;
   }
   void display() {
      System.out.println("Student Details");
      System.out.println("Name of Student: " + name);
      System.out.println("Roll No: " + roll_no);
      System.out.println("Academic score: " + score);
   }
}
class Result extends Student implements Sports {
  int score;
  Scanner sc = new Scanner(System.in);
   Result(String name, int roll, int m1) {
      super(name, roll, m1);
   }
   public void sports_getData() {
      System.out.print("Enter the Score you have obtain in sports");
      score = sc.nextInt();
   }
   public void sports_dispData() {
      System.out.println("Sports score:" + score);
   }
}
class co3_5 {
   public static void main(String args[]){
      String name;
      int roll,mark;
      Scanner sc = new Scanner(System.in);
      System.out.print("enter the name:");
      name = sc.nextLine();
      System.out.print("enter the roll no:");
      roll = sc.nextInt();
      System.out.print("enter the academic score:");
      mark = sc.nextInt();
      Result re = new Result(name,roll,mark);
      re.sports_getData();
      re.display();
      re.sports_dispData();
   }
}
```

OUTPUT

```
PS D:\MCA\sem2\Java> javac co3_5.java
PS D:\MCA\sem2\Java> java co3_5
enter the name:ram
enter the roll no:101
enter the academic score:92
Enter the Score you have obtain in sports89
Student Details
Name of Student: ram
Roll No: 101
Academic score: 92
Sports score:89
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 15

AIM:

Create an interface having prototypes of functions area() and perimeter(). Create two classes Circle and Rectangle which implements the above interface. Create a menu driven program to find area and perimeter of objects.

ALGORITHM:

Step 1: Start
Step 2: Define an Interface Shape
  2.1 Declare two methods:
    - double area()
    - double perimeter()
Step 3: Define Class Circle Implementing Shape
  3.1 Declare radius as a data member.
  3.2 Define constructor to initialize radius.
  - Implement:
    - area() as π * radius * radius
    - perimeter() as 2 * π * radius
Step 4: Define Class Rectangle Implementing Shape
  4.1 Declare length and width as data members.
  4.2 Define constructor to initialize length and width.
  4.3 Implement:
    - area() as length * width
    - perimeter() as 2 * (length + width)
Step 5: In the main() Method
  5.1 Create Scanner object for input.
  5.2 Declare choice variable.
  5.3 Repeat indefinitely using a while(true) loop:
    - Display menu options:
      - Circle
      - Rectangle
      - Exit
    - Prompt user to enter choice.
  5.4 Use switch-case to handle choice:
    - Case 1:
      - Ask user to enter radius
      - Create a Circle object
      - Display:
        circle.area()
        circle.perimeter()
    - Case 2:
      - Ask user to enter length and width
      - Create a Rectangle object
      - Display:
        rectangle.area()
        rectangle.perimeter()
    -Case 3:
      - Print exit message

      - Exit the loop using return
      -Default:

- Print "Invalid choice" message

Step 6: End

SOURCE CODE:

```
import java.util.Scanner;
interface Shape {
   double area();
   double perimeter();
}
class Circle implements Shape {
   double radius;
   Circle(double radius) {
      this.radius = radius;
   }
   public double area() {
      return Math.PI * radius * radius;
   }
   public double perimeter() {
      return 2 * Math.PI * radius;
   }
}
class Rectangle implements Shape {
   double length;
   double width;
   Rectangle(double length, double width) {
      this.length = length;
      this.width = width;
   }
   public double area() {
      return length * width;
   }
   public double perimeter() {
      return 2 * (length + width);
   }
}
public class co3_6 {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int choice;
      while(true) {
         System.out.println("\n--- Menu ---");
         System.out.println("1. Calculate area and perimeter of Circle");
         System.out.println("2. Calculate area and perimeter of Rectangle");
         System.out.println("3. Exit");
         System.out.print("Enter your choice: ");
         choice = sc.nextInt();
         switch (choice) {


case 1:
```

```
            System.out.print("Enter radius of the circle: ");
            double radius = sc.nextDouble();
            Circle circle = new Circle(radius);
            System.out.println("Area of Circle: " + circle.area());
            System.out.println("Perimeter of Circle: " + circle.perimeter());
            break;
case 2:
            System.out.print("Enter length of the rectangle: ");
            double length = sc.nextDouble();
            System.out.print("Enter width of the rectangle: ");
            double width = sc.nextDouble();
            Rectangle rectangle = new Rectangle(length, width);
            System.out.println("Area of Rectangle: " + rectangle.area());
            System.out.println("Perimeter of Rectangle: " + rectangle.perimeter());
            break;
case 3:
           System.out.println("Exiting program...");
           return;
default:
            System.out.println("Invalid choice! Please try again.");
             }
        } } }
```

OUTPUT:

```
PS D:\MCA\sem2\Java> javac co3_6.java
PS D:\MCA\sem2\Java> java co3_6

  --- Menu ---
  1. Calculate area and perimeter of Circle
  2. Calculate area and perimeter of Rectangle
  3. Exit
  Enter your choice: 1
  Enter radius of the circle: 4
  Area of Circle: 50.26548245743669
  Perimeter of Circle: 25.132741228718345

  --- Menu ---
  1. Calculate area and perimeter of Circle
  2. Calculate area and perimeter of Rectangle
  3. Exit
  Enter your choice: 2
  Enter length of the rectangle: 3
  Enter width of the rectangle: 4
  Area of Rectangle: 12.0
  Perimeter of Rectangle: 14.0

  --- Menu ---
  1. Calculate area and perimeter of Circle
  2. Calculate area and perimeter of Rectangle
  3. Exit
  Enter your choice: 3
  Exiting program...
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 16

AIM:

Prepare bill with the given format using calculate method from interface.
Order No.

ALGORITHM:

Step 1: Start
Step 2: Define an Interface Bill
  2.1 Declare a method: void calculate()
Step 3: Define a Class Product that Implements Bill
  3.1 Declare data members:
    -int productId, int quantity
    -String name
    -float unitPrice, float total
  3.2 Define method getData() to:
    -Take input for all product details from the user.
  3.3 Define method calculate() to:
    - Compute total = quantity * unitPrice.
  3.4 Define method display() to:
    - Print product details.
  3.5 Define method getTotal() to:
    - Return the value of total.
Step 4: In the main() Method
  4.1 Create Scanner object for user input.
  4.2 Prompt user to enter:
    - Order number → orderNo
    - Number of products → n
  4.3 Create an array of Product objects of size n.
  -Initialize netAmount = 0.
Step 5: For Each Product
  5.1 Create a new Product object.
  - Call:
    - getData() to read product info.
    - calculate() to compute total price.
  5.2 Add each product's total to netAmount.
Step 6: Print the Bill
  - Print:
    - Order number and current date.
    - Table header for product info.
    - Product details by calling display() on each product.
  - Print the net amount using formatted output.
Step 7: End

SOURCE CODE

```
import java.util.*;

interface Bill {
```

```
    void calculate();
}
class Product implements Bill {
    int productId;
    String name;
    int quantity;
    float unitPrice;
    float total;
    void getData() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Product ID: ");
        productId = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Product Name: ");
        name = sc.nextLine();
        System.out.print("Enter Quantity: ");
        quantity = sc.nextInt();
        System.out.print("Enter Unit Price: ");
        unitPrice = sc.nextFloat();
    }
    public void calculate() {
        total = quantity * unitPrice;
    }
    void display() {
        System.out.println(productId+"\t\t"+name+"\t"+quantity+"\t\t"+unitPrice+"\t"+total);
    }
    float getTotal() {
        return total;
    }
}

public class co3_7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Order Number: ");
        int orderNo = sc.nextInt();
        System.out.print("Enter Number of Products: ");
        int n = sc.nextInt();
        Product[] products = new Product[n];
        float netAmount = 0;
        System.out.println("\nEnter product details:\n");
        for (int i = 0; i < n; i++) {
            products[i] = new Product();
            products[i].getData();
            products[i].calculate();
            netAmount += products[i].getTotal();
            System.out.println();
        }
```

```
        System.out.println("\nOrder No.: " + orderNo);
        System.out.println("Date: "+ java.time.LocalDate.now());
        System.out.println("-------------------------------------------------------");
        System.out.println("ProductId\tName\tQuantity\tUnit\tPriceTotal");
        System.out.println("-------------------------------------------------------");
        for(Product p : products) {
            p.display();
        }
        System.out.println("-------------------------------------------------------");
        System.out.printf("%40s %-10.2f\n", "Net. Amount", netAmount);
    }
}
```

OUTPUT:

```
PS D:\MCA\sem2\Java> javac co3_7.java
PS D:\MCA\sem2\Java> java co3_7
Enter Order Number: 1001
Enter Number of Products: 2

Enter product details:

Enter Product ID: 101
Enter Product Name: A
Enter Quantity: 2
Enter Unit Price: 25

Enter Product ID: 102
Enter Product Name: B
Enter Quantity: 1
Enter Unit Price: 100


Order No.: 1001
Date: 2025-05-04
-------------------------------------------------------
ProductId       Name    Quantity        Unit    PriceTotal
-------------------------------------------------------
101             A       2               25.0    50.0
102             B       1               100.0   100.0
-------------------------------------------------------
                                Net. Amount 150.00
PS D:\MCA\sem2\Java>
```

Result:

Successfully executed

PROGRAM NO 17

AIM:

Create a Graphics package that has classes and interfaces for figures Rectangle, Triangle, Square and Circle. Test the package by finding the area of these figures.

ALGORITHM:

Step 1: Start
Step 2: Define a Package Graphics
Step 3: Inside the Package, Define an Interface inter_graph
3.1 Declare abstract methods:
- int rect(int l, int w) → for area of rectangle
- double tria(double b, double h) → for area of triangle
- float squ(int a) → for area of square
- double cir(double r) → for area of circle

Step 4: Define a Class area that Implements inter_graph
4.1 Implement each method:
- rect(l, w) → return l * w
- tria(b, h) → return 0.5 * b * h
- squ(a) → return a * a
- cir(r) → return π * r * r

Step 5: Create Another File TestGra.java Outside the Package
5.1 Import Graphics.area class
-Define class TestGra with main() method

Step 6: In main()
6.1 Create object g of class area
6.2 Call methods using the object:
- g.rect(4,5) and print result
- g.tria(3,4.5) and print result
- g.squ(5) and print result
- g.cir(4.5) and print result

Step 7: End

SOURCE CODE

area.java

```
  package Graphics;
interface inter_graph{
  public int rect(int l,int w);
  public double tria(double b,double h);
  public float squ(int a);
  public double cir(double r);
 }
 public class area implements inter_graph{
  public int rect(int l,int w){
   return l*w;
   }
```

```
 public double tria(double b,double h){
    return 0.5*b*h;
   }
  public float squ(int a){
    return a*a;
   }
  public double cir(double r){
     return Math.PI*r*r;
   }
 }
```

TestGra.java

```
    import Graphics.area;
    public class TestGra{
      public static void main(String[] args){
        area g =new area();
        System.out.println("area of rectangle:"+g.rect(4,5));
        System.out.println("area of triangle:"+g.tria(3,4.5));
        System.out.println("area of Square:"+g.squ(5));
        System.out.println("area of circle:"+g.cir(4.5));
      }
     }
```

OUTPUT:

```
PS D:\MCA\sem2\Java\Graphics> javac TestGra.java
PS D:\MCA\sem2\Java\Graphics> java TestGra
area of rectangle:20
area of triangle:6.75
area of Square:25.0
area of circle:63.61725123519331
PS D:\MCA\sem2\Java\Graphics>
```

Result:

Successfully executed

PROGRAM NO 18

AIM:

Create an Arithmetic package that has classes and interfaces for the 4 basic arithmetic operations. Test the package by implementing all operations on two given numbers.

ALGORITHM:

Step 1: Start
Step 2: Define a Package Arithmetic
Step 3: Inside the Package, Define an Interface inter_arith
  3.1 Declare the following abstract methods:
    -float add(int a, int b)
    -float sub(int a, int b)
    -float mult(int a, int b)
    -float div(int a, int b)
    -float rem(int a, int b)
Step 4: Define a Class oprtn that Implements inter_arith
  4.1 Implement all the methods:
    - add(a, b) → return a + b
    - sub(a, b) → return a - b
    - mult(a, b) → return a * b
    - div(a, b) → return a / b
    - rem(a, b) → return a % b
Step 5: Create Another File TestArit.java Outside the Package
  5.1 Import the Arithmetic package
  5.2 Define class TestArit with the main() method
Step 6: In the main() Method
  6.1 Create object obj of class oprtn
  6.2 Call and display the results of:
    - obj.add(45, 4) → Print result
    - obj.sub(34, 18) → Print result
    - obj.mult(9, 4) → Print result
    - obj.div(45, 4) → Print result
    - obj.rem(45, 10) → Print result
Step 7: End

SOURCE CODE

oprtn.java

```
package Arithmetic;
interface inter_arith{
  public float add(int a,int b);
  public float sub(int a,int b);
  public float mult(int a,int b);
  public float div(int a,int b);
  public float rem(int a,int b);
```

```
}
 public class oprtn implements inter_arith{
  public float add(int a,int b){
      return a+b;
   }
  public float sub(int a, int b){
     return a-b;
   }
  public float mult(int a, int b){
    return a*b;
  }
  public float div(int a, int b){
   return a/b;
  }
  public float rem(int a,int b){
   return a%b;
  }
  }
```

TestArith.java

```
import Arithmetic.*;
public class TestArit{
 public static void main(String[] args){
 oprtn obj = new oprtn();
 System.out.println("addition:"+obj.add(45,4));
System.out.println("substraction:"+obj.sub(34,18));
System.out.println("multipication:"+obj.mult(9,4));
  System.out.println("division:"+obj.div(45,4));
 System.out.println("reminder:"+obj.add(45,10));
}
}
```

OUTPUT:

```
PS D:\MCA\sem2\Java> cd Arithmetic
PS D:\MCA\sem2\Java\Arithmetic> javac TestArith.java
PS D:\MCA\sem2\Java\Arithmetic> java TestArith
addition:49.0
substraction:16.0
multipication:36.0
division:11.0
reminder:55.0
PS D:\MCA\sem2\Java\Arithmetic>
```

Result:

Successfully executed